

Automatic Dockerfile Generation with Large Language Models

Jun Lyu

State Key Laboratory for Novel
Software Technology, Software
Institute, Nanjing University
Nanjing, China
junlyu@nju.edu.cn

He Zhang*

State Key Laboratory for Novel
Software Technology, Software
Institute, Nanjing University
Nanjing, China
hezhang@nju.edu.cn

Yusong Yuan

State Key Laboratory for Novel
Software Technology, Software
Institute, Nanjing University
Nanjing, China
yusong.yuan@outlook.com

Lanxin Yang

State Key Laboratory for Novel
Software Technology, Software
Institute, Nanjing University
Nanjing, China
lxyang@nju.edu.cn

Yue Li

State Key Laboratory for Novel
Software Technology, Software
Institute, Nanjing University
Nanjing, China
liyue@nju.edu.cn

Manuel Rigger*

National University of Singapore
Singapore, Singapore
rigger@nus.edu.sg

Abstract

Project maintainers on platforms like GitHub usually provide a document (e.g., a README) describing how to build and use their software, whose instructions, often written in natural language, might sometimes be ambiguous or platform-specific. This task can be eased by containerization, which allows developers to define the build environments required for their software and to enable automatic builds. Docker is the leading platform in this field and relies on Dockerfiles for correct execution. Practitioners who want to use and build projects that do not include a Dockerfile need to develop it themselves. They often need to perform multiple builds by trial and error to discover the correct build steps and build dependencies (e.g., third-party library necessary for software builds). To address this challenge, we propose a novel approach and tool called Dockerfile Refinement and Fixing Tool (DRAFT), which can automatically generate a working Dockerfile by a Large Language Model (LLM). The core idea of DRAFT is that it emulates developers' behavior iteratively. Specifically, DRAFT is used to iteratively generate prompts to instruct the LLM to generate patches for an initially generated Dockerfile until it executes successfully. Subsequently, it attempts to optimize the Dockerfile to reduce the size of the image it builds. We evaluated DRAFT on 35 projects and compared it with the Dockerfiles generated by Copilot (based on ChatGPT 4.0), Actions_remaker, Repo2docker, Boxing, and Starter in five different languages (C, Java, Go, Python, and Ruby). The evaluation results show that DRAFT can successfully generate Dockerfiles for 28 projects, resulting in a success rate of 80%, which is much higher than all compared tools. The time consumed by DRAFT on generating Dockerfiles is 19.57 minutes at maximum with the median at 33.78s. Using DRAFT can reduce image size by 67.14% and build time by 3.62%. DRAFT relieves practitioners from the burden of manually creating build specifications.

*Corresponding author: He Zhang and Manuel Rigger.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ICSE '26, Rio de Janeiro, Brazil

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2025-3/26/04
<https://doi.org/10.1145/3744916.3773162>

CCS Concepts

• Software and its engineering → Software maintenance tools.

Keywords

Build Tool, Large Language Mode, Dockerfile

ACM Reference Format:

Jun Lyu, He Zhang, Yusong Yuan, Lanxin Yang, Yue Li, and Manuel Rigger. 2026. Automatic Dockerfile Generation with Large Language Models. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3744916.3773162>

1 Introduction

Software building is a critical activity for developers and maintainers of a project [35]. Building a new project from platforms like GitHub [20] can be challenging, and even more so for users unfamiliar with the project, given that many projects require multiple environmental dependencies [58]. The project owner usually provides an instruction file (e.g., in a README) on how to build the software. However, the instructions in the README are entangled with natural language, making it error-prone to follow them. In addition, these instructions might differ depending on the platform, further complicating the use of them. In many other projects, the situation is worse [7], as required software dependencies might not explicitly be described, or instructions are implicit. This requires users to spend much time trying to build the software of interest.

Dockerfile creation remains challenging due to significant documentation inconsistencies across projects. In mature projects, verbose deployment-specific documentation risks obscuring critical steps, leading to misextraction and build errors. Conversely, early-stage projects frequently provide incomplete build context, causing dependency gaps and version conflicts during execution. Both scenarios force engineers to manually distill core build procedures, a time-consuming and error-prone process. Fig. 1 shows the installation instruction file from the FFmpeg project.¹ The first two lines illustrate the use of `./configure` to configure FFmpeg to build different targets. The third line describes what is required to build FFmpeg (GNU Make 3.81 or later). The fourth line describes

¹<https://github.com/FFmpeg/FFmpeg/blob/master/INSTALL.md>

Install FFmpeg

1. Type `./configure` to create the configuration. A list of configure options is printed by running `configure --help`.
2. configure can be launched from a directory different from the FFmpeg sources to build the objects out of tree. To do this, use an absolute path when launching `configure`, e.g. `/ffmpegdir/ffmpeg/configure`.
3. Then type make to build FFmpeg. `GNU Make 3.81` or later is required.
4. Type `make install` to install all binaries and libraries you built.

Figure 1: Installation Instruction from FFmpeg Project (Commit ID: 440e08e)

building FFmpeg using the build command (`make install`). We tried to build FFmpeg according to its instructions in a new environment (Ubuntu 22.04). We first executed `./configure` and the build system indicated missing `nasm/yasm` dependencies, which are assembly compilers [40, 62] that are necessary dependencies for building FFmpeg, but are not specified.

Containerization eases the building of software, as a container specifies and manages its dependencies. Containers can also isolate software, preventing it from interfering with one another and ensuring that dependencies are contained within the container. Since containers have their own allocated computing resources, predictable performance can be achieved [2]. Docker is one of the most popular tools in this field [39, 54]. Docker requires project maintainers or users to develop a Dockerfile [13], which contains a set of commands that, when executed, builds a Docker image that can be executed in one or more lightweight virtual machines (Docker containers).

Some software projects include Dockerfiles developed by maintainers, but they are often not kept up to date, and many Dockerfiles cannot be built correctly [7]. Cito et al. [7] pointed out that there are still quality issues with the Dockerfile in the 100 projects with the most GitHub stars, for example, the pinning of the base image and the lack of dependencies. They also evaluated a sample of 560 open-source projects on GitHub and found that more than a third of these projects had Dockerfiles that could not be built without errors. In addition, it is common for developers to spend between 20 minutes and two hours constructing a Dockerfile for a Gist [19]. Some Gists even fail to build a valid Dockerfile [25]. In summary, while many projects lack Dockerfiles, even those with Dockerfiles might not necessarily work or be up to date.

Various approaches have been proposed to automatically generate Dockerfiles. AI chatbots have demonstrated high effectiveness for many tasks, and can also be prompted to generate Dockerfiles (e.g., Copilot [9]), although no specific approach based on them has been proposed. Copilot, an AI chatbot, can be used to generate Dockerfiles, and it can support projects in multiple languages. In addition, some approaches or tools have been proposed that are programming-language specific, such as `Actions_remaker` [1], `Starter` [55], `Generator-docker` [18], `Humpback` [27], `Repo2docker` [31], `Boxing` [4], and a Dockerfile generation method based on deep learning [50]. These methods and tools are designed for specific languages, hence it is difficult to generalize them to be language-agnostic. Furthermore, they cannot identify dependencies not mentioned in the instruction of the project (e.g., `nasm/yasm` in Fig. 1), and practitioners still need to invest effort to generate a working Dockerfile. This is because these methods are

solely based on software build instructions, failing to account for those not explicitly documented in the software build instructions. Hence, these methods struggle to generate a working Dockerfile.

We propose a novel automatic Dockerfile generation approach called Dockerfile Refinement and Fixing Tool (DRAFT), which is language-agnostic. The goal behind DRAFT is to generate Dockerfiles through an LLM (e.g., ChatGPT 3.5 or Deepseek-v3). Doing so in a straightforward manner would often fail, as each LLM-generated Dockerfile may be potentially erroneous or incomplete—prompting ChatGPT 4.0 without using DRAFT results in only 31.43% working Dockerfiles, cf. Section 4.2. To tackle this, DRAFT is designed with an iterative refinement process to keep refining Dockerfiles until a successful buildable image Dockerfile has been generated for the project. DRAFT automatically generates prompts by executing each Dockerfile while retaining its error logs. It generates multiple candidate patches (new Dockerfiles) for each error by *error-driven prompts* and *context-driven prompts*, and then selects a patch (a Dockerfile). The key idea behind these two kinds of repair is to generate patches from the errors and the project build instructions, respectively. DRAFT continues with applying repairs until the Dockerfile executes without an error—in which case we deem the build to be successful—or when it exceeds a predefined attempt threshold. The users can provide a command or script that can validate whether the build indeed has been successful. In addition, after a project successfully builds, we attempt to reduce its image size through interactions with the LLM incrementally.

We evaluated DRAFT on 35 well-known and widely used projects in five languages (C, Java, Go, Python, and Ruby) and compared it with the latest publicly available state-of-the-art tools (Copilot [9], `Actions_remaker` [1], `Repo2docker` [31], `Boxing` [4], and `Starter` [55]). Using projects written in five different languages as benchmarks, we compared DRAFT with Copilot and `Action_remaker`, `Repo2docker`, `Boxing`, and `Starter`. The evaluation results show that DRAFT successfully generated Dockerfiles for 28 projects and failed for seven (three projects failed testing). It outperformed all the tools compared in terms of success rate in generating Dockerfiles. The time consumption for DRAFT to generate Dockerfiles ranges from 5.60s to 19.57 minutes, with an average of 195.69s and a median of 33.78s. DRAFT demonstrates its effectiveness in generating Dockerfiles. In addition, using DRAFT can reduce image size by 67.14% and build time by 3.62%. DRAFT can benefit practitioners by reducing the workload associated with developing build specifications. DRAFT addresses challenges due to significant variability in project documentation. In the projects we evaluated, the size range of the documentation is 31 (`sinbox`)–841 (`redis`) lines. DRAFT can generate Dockerfiles for 80% of projects, including `sing-box`, which has the sparsest documentation, and `redis`, which has the most detailed documentation. DRAFT’s ability to generate Dockerfiles is not affected by whether project documentation is detailed or sparse. In addition, this work shows that a conceptually simple LLM-based technique can outperform existing state-of-the-art approaches.

2 Motivating Example

In this section, we explain the challenge of building Dockerfiles based on the scenario in which a practitioner would build FFmpeg,

a popular video and audio command line tool, with libx264, a video encoder, enabled building on the example discussed based on Fig 1.

Developing Dockerfiles. Developing a Dockerfile usually requires specifying the build dependencies and execution environment of the target software. As shown in Fig. 2, the process begins with specifying the software’s base image (the latest version of Ubuntu, line 1) via “FROM”. Subsequent steps, defined via “RUN” commands, include updating the package list to ensure access to the latest package information and installing the necessary build dependencies for the target software (line 2). The target software is then copied into the image (“COPY”, line 3), sets its working directory (“WORKDIR”, line 4), and executes the commands (“RUN”, line 5). Running Docker on this Dockerfile results in an error (line 6). The error indicating the use of “enable libx264” requires explicitly setting the artifact’s license to GPL [21]. Even after resolving this, a new error occurs due to a missing “x264” dependency (lines 7–8). Such iterative manual refinement and troubleshooting using platforms like Stack Overflow [53] is laborious.

Generating Dockerfile by Copilot. Fig 3 shows the Copilot-generated Dockerfile that supports libx264. We took the repository address as input and instructed Copilot to generate a Dockerfile based on it. Different from Fig 2, Copilot enables partial dependencies to be installed for the project (line 2). The Dockerfile generated according to it avoids errors due to missing nasm/yasm. However, as shown in Fig 2, this Dockerfile is still missing “-enable-gpl” (line 6), causing a build failure.

3 Approach

To automate the generation of Dockerfiles, we propose a new approach called Dockerfile Refinement and Fixing Tool (DRAFT). The core idea of DRAFT is to emulate a human trial-and-error approach by generating and then incrementally refining a Dockerfile through repeated interaction with the LLM as well as Docker. Specifically, DRAFT uses an LLM to generate an initial Dockerfile for the project, which may be incorrect or incomplete. If the Dockerfile is incorrect, Docker will indicate the error in its build logs, which we can utilize for refinement. DRAFT executes the initial Dockerfile and tries to locate errors from its build logs. DRAFT automatically generates prompts to instruct the LLM to generate candidate patches (Dockerfiles) for each error. It repeats the execution of the Dockerfile, locating build errors, and repairing them, until there are no errors in the Dockerfile execution. Additionally, DRAFT can continue to be used to optimize the Dockerfile (e.g., image size).

```
1. FROM ubuntu:latest
2. RUN apt-get update && apt install libx264-dev
3. COPY . /ffmpeg
4. WORKDIR /ffmpeg
5. RUN ./configure --enable-libx264 && make && make install
6. ERROR: libx264 is gpl and --enable-gpl is not specified.
# Revising the Dockerfile
7. RUN ./configure --enable-libx264 --enable-gpl && make && make install
8. ERROR: x264 not found using pkg-config
```

Figure 2: Developing a Dockerfile to Support X264 for FFmpeg Project (Commit ID: 440e08e). Errors Are in Red.

```
1. FROM ubuntu:20.04
2. RUN apt-get update && apt-get install -y build-essential yasm libx264-dev
3. WORKDIR /app
4. RUN git clone https://github.com/FFmpeg/FFmpeg.git
5. RUN cd FFmpeg && \
6. ./configure --enable-libx264 && make && make install
```

Figure 3: Generating a Dockerfile by Copilot to Support x264 for FFmpeg Project (Commit ID: 440e08e)

An overview of DRAFT is shown in Fig 4. DRAFT accepts a project repository link as an input and outputs a Dockerfile. It generates the initial Dockerfile for the project (step ①). DRAFT executes the initial Dockerfile and records its build log. Then, it locates build errors through the build log (step ②). DRAFT automatically generates prompts to instruct the LLM to generate candidate patches (follow-up Dockerfiles) based on different strategies focusing on aspects such as error logs or project context (step ③), based on the intuition that a diverse range of patches can maximize the chances of successfully resolving an error. DRAFT generates Dockerfiles by repeating the above steps (steps ②–③). We consider a generation attempt for a Dockerfile to be successful if a version of the project is successfully built in the resulting Docker image. DRAFT emulates typical human behavior, where, in order to resolve a build error, developers are likely to try different strategies based on error logs as well as documentation.

3.1 Pre-processing

In this section, we present the pre-processing that DRAFT performs on subjects before generating the initial Dockerfile (step ①). This step is executed only once, and it is the starting point of our approach.

Project context. DRAFT expects users to provide project context. Specifically, users can specify up to two documents from the project that they deem most suitable in providing information on how to build the project. Typically, this could be a Readme and an Install file. These files are usually located in the root directory of the project. DRAFT uses the above as part of the input for generating the initial Dockerfile and also as part of the input of *context-driven prompts*.

Generating initial Dockerfile. DRAFT uses pre-processing results to guide an LLM to output the initial Dockerfile. The initial Dockerfile might fail building; if so, we repair it by following the subsequently described steps. Through predefined message templates, DRAFT communicates with the LLM.

3.2 Iterative Refinement

This section illustrates how DRAFT iteratively refines Dockerfiles. DRAFT generates Dockerfiles by first locating the build error, generating prompts to repair this error, and selecting candidate patches (Dockerfiles) that might fix the errors. However, candidate patches may generate new errors, thus, these steps might be repeated multiple times (steps ②–③). Step ② starts with an initially-generated or partially-repaired Dockerfile that has a build error, preventing the build from completing (e.g., line 9 in Fig 2). Build logs tend to record a large amount of information concerning the various stages of the build process, such as the detailed process of task execution.

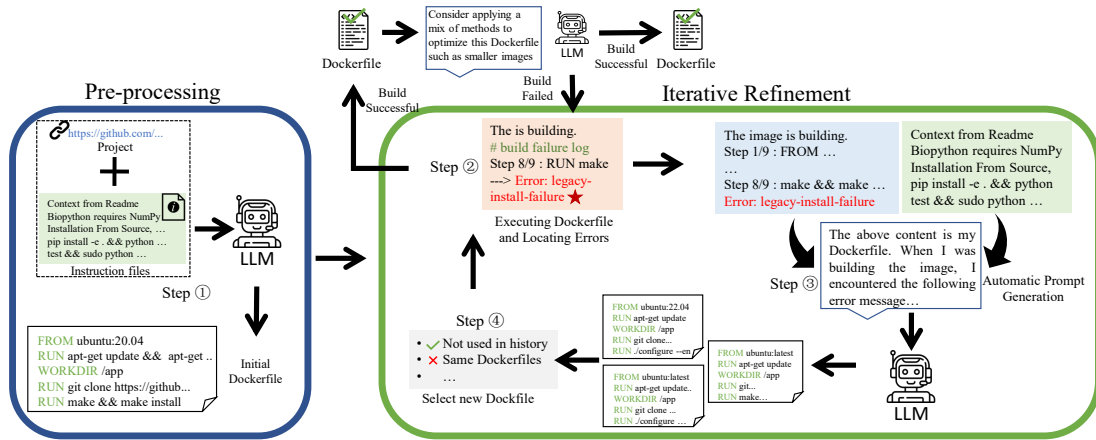


Figure 4: Overview of DRAFT

Finding the root cause in build logs allows for the repair of build errors. DRAFT logs the build and finds the build error.

Locating errors. DRAFT identifies errors by detecting error keywords in build logs and leveraging LLMs to analyze the logs captured upon completion of a Dockerfile execution. (step ②).

We used LLM-based error localization as a general approach. A localization method that is always applicable is to intercept the build log at the end and locate the error using an LLM. Specifically, we prompt the LLM to locate the error given the end of the build log. In a Docker image build, the build log is usually terminated when a build error occurs, and the end of the build log usually contains a paragraph of error logs output by the build system and error logs output by Docker. Hence, DRAFT takes a part of the build logs. However, typically, the log output cannot directly be used for repair, because the error output by Docker does not directly point to the reason for the build failure. The downside of the approach is that interaction with the LLM is costly.

As an optimization, we also introduced a keywords-based approach. The key observation we have made is that builds usually have similar error descriptions in the build system when the build fails, allowing the error to be located by keywords. DRAFT uses keywords to locate errors such as “not found” and “not specified”. In the event of a build error, the build system outputs an error message by line. DRAFT traverses each line matching the keyword and processes the line where the keyword is located. For example, for the keywords “not specified”, DRAFT obtains the word that precedes these. DRAFT saves the words so that when repairing the error, it can check if the LLM is repairing it for this error. This localization technique fails if the build fails with uncommon error messages. While not general, it can effectively speed up error localization. Fixed-pattern error localization method (by keywords used by DRAFT) is difficult to deal with in all cases. Using keywords cannot locate all errors correctly, because build errors are varied for each project [37]. Therefore, DRAFT was designed to intercept build logs and allow an LLM to locate errors.

Automatic prompt generation. DRAFT repairs errors by using two strategies: *error-driven prompts* and *context-driven prompts* (step ③). DRAFT continues to work in the event of a build failure until it

generates a Dockerfile that can be built successfully. DRAFT alternates between the two prompt types when a build error occurs. This means that DRAFT tries to generate multiple patches (Dockerfiles) using *error-driven prompts*, and when the patch fails, it tries to use *context-driven prompts*. Each of the two repair strategies generates up to three patches.

The first method is *error-driven prompts*. Specifically, DRAFT uses the latest Dockerfile and the intercepted build logs to instruct an LLM to repair the error. DRAFT inputs build logs and the Dockerfile in which the errors occurred, and instructs the LLM that this is the Dockerfile it generated but an error occurred, instructing it to generate patches (Dockerfiles) for the errors. It generates three patches (Dockerfiles) for a build error in each of the two strategies. DRAFT generates multiple candidate patches, because the LLM may not have correctly understood the build error and could not correctly generate a patch for it.

When *error-driven prompts* fails, DRAFT attempts *context-driven prompts*. DRAFT uses the project context, the latest Dockerfile, and the intercepted build logs to instruct an LLM to generate a new Dockerfile. Specifically, DRAFT adds the project context to the input of LLM and instructs it to refer to the context before generating patches (Dockerfiles) against the build logs and the Dockerfile where the error occurred. It is difficult to obtain the information of the project context through the build log.

After obtaining patches from *context-driven prompts* and *error-driven prompts*, DRAFT determines whether and which patch to apply based on four rules. The strategy of DRAFT for selecting patches is, firstly, that this patch has not been used before (Rule 1), and secondly, if the same patch (syntactically identical) is generated in a single repair, then this same patch is selected (Rule 2). DRAFT generates three patches in a single repair that originate from the same prompt. This is to account for the stochastic nature of LLMs. If multiple responses are consistent, they indicate the most likely plausible response. Thirdly, DRAFT locates the core statements in the error log by keywords, it keeps the words and selects the Dockerfile by matching the words (Rule 3). Consider the following error example: “ERROR: libx264 is gpl and -enable-gpl is not specified.” “ERROR” is a keyword; DRAFT keeps the core words: “libx264”, “gpl”, “-enable-gpl”, and “specified”. DRAFT filters out stop words

Table 1: Methods to Optimizing Dockerfiles

Methods	Description
Smaller base image	The base image is the starting point for a Docker image. Using a smaller base image reduces the overall size of the final image.
Consolidating commands	Each command in the Dockerfile (e.g., RUN, COPY) creates a new layer, and too many layers can lead to increased image size and slower build times. It is important to merge directives wherever possible to minimize the number of layers.
Limit installed packages	This practice helps reduce the size of the final Docker image by installing a limited number of dependencies and removing them after the build.
Multi-stage builds	Multi-stage builds allow developers to create multiple images from a single Docker file. This allows for the separation of build and run environments, the ability to reduce image size, and improved performance.
Docker ignore file	The .dockerignore file specifies the files and directories that should be ignored during the build process. This can greatly reduce the amount of data copied to the image, reducing build speed and speeding up the build.

```
{Dockerfile} The above content is my Dockerfile. When I was building the
image, I encountered the following error message: '{error}'. Please analyze
the above information and generate a new Dockerfile for me. You output
in the format below:
'''Dockerfile
# Write the contents of your output Dockerfile below.
'''
```

Figure 5: Prompts Example

such as “is”. Patches are then selected by matching these words in Dockerfiles. Finally, if none of the previous three rules can select a patch, DRAFT uses LLMs to select a patch (Rule 4). DRAFT takes the generated patch, the error logs, and the Dockerfile with errors as inputs and instructs the LLMs to select a patch.

Prompts Format. The key idea behind prompts is to convert the steps ①–③ into succinct instructions and inputs (e.g., error logs). All prompts consist of instructions as well as given inputs. An example of prompts is shown in Fig 5. All prompts can be found in the supplementary material.

As shown in Fig 5, this prompt is used every time DRAFT instructs ChatGPT to generate a new Dockerfile (patch). At the start of the prompt is inputs, DRAFT includes as part of the prompt the Dockerfiles that failed to execute, and states that it failed and got an error log when executing this Dockerfile to build the image. DRAFT then prompts ChatGPT to generate a new Dockerfile (patch) based on this information, which are the instructions. At the prompt’s end, DRAFT instructs ChatGPT to output a Dockerfile that conforms to the specified format.

3.3 Optimizing Dockerfiles

The Dockerfile generated from DRAFT may have some limitations in terms of adaptability to real-world applications, such as excessive build image size and build time. Therefore, DRAFT is designed with additional prompts to optimize the Dockerfile automatically.

We have summarized experiences from practitioners optimizing Dockerfiles and designed multiple prompts. We started by searching for posts on Stack Overflow via the keyword “Dockerfile optimize”. We read through the top 50 posts to determine the methods that can be used to optimize Dockerfiles, and the results are shown in Table 1. We extended DRAFT to try to optimize Dockerfiles using a similar high-level methodology. Specifically, we designed multiple prompts and instructed ChatGPT to optimize Dockerfiles using each of the five optimization methods, and we used DRAFT to fix optimized but unbuildable Dockerfiles. We count how much image size and build time each method optimizes. In addition, we explore the extent to which these five methods can reduce image size and build time.

3.4 Implementation

In this section, we further illustrate the implementation details of DRAFT.

Project context. In our evaluation, we provided DRAFT with only the Readme file included in each project. Readmes are usually located in the root directory of the project, and typically contain build instructions of varying detail. During preprocessing, DRAFT reads these files and removes line breaks as they do not contain any semantic information needed for building.

Keywords and locating errors. Ten keywords were selected to capture common build failures (i.e., “error”, “unknown”, “missing”, “no such file or directory”, “failed to execute”, “execution failed”, “file not found”, “not specified”, “unable to locate package”, “no matching distribution found for”). While keyword-based detection is efficient for known error patterns, it is inherently limited in handling novel or complex errors. To mitigate this, we augmented the system with an LLM-based approach that analyzes the error log contextually. Given the token limitations of models like ChatGPT 3.5, we preprocess the build log by extracting the last 50 lines. While 50 is a reasonable but arbitrary limit, typically, the whole log contains much irrelevant output in addition to the error logs. In our practice, 50 lines is what we consider an appropriate build log length. DRAFT takes the 50-line log as input for the step ③.

4 Evaluation

We evaluate the effectiveness and efficiency of DRAFT on practical and widely used software systems. The evaluation is driven by the following questions (Q).

Q1: How effective is DRAFT compared to state-of-the-art tools in generating the Dockerfile?

Goal. This question explores the effectiveness of DRAFT in generating correct Dockerfiles. Specifically, we aim to determine whether the Dockerfiles can build the Docker image correctly.

Q2: How efficient is DRAFT in generating a Dockerfile?

Goal. This question explores the time consumption of DRAFT in generating Dockerfiles.

Q3: How do different components contribute to the effectiveness of DRAFT?

Goal. The main components of DRAFT are the pre-processing method and automatic prompt generation. We explore whether these two approach designs are effective separately. To evaluate Q3, we performed two experiments.

Q4: Could DRAFT be able to optimize the Dockerfiles it generates so as to reduce the image size and build time?

Goal. Dockerfiles created by DRAFT may be underperforming, *e.g.*, the images are too large and waste compute resources. We explore whether DRAFT can be used to optimize Dockerfiles.

4.1 Experiment Setting

Subjects. We selected 35 projects based on the following characteristics: they are C, Go, Java, Python, and Ruby projects, they can build successfully, and they are highly mature and popular projects. We selected the five languages with reference to the work [59] and TIOBE ranking [56], which shows that Python, C, Java, and Go are the most popular languages. In addition, we chose the less popular language Ruby to reduce bias. To identify them, we performed two steps. In step one, a GitHub search targeted repositories using C, Go, Java, Python, and Ruby. In step two, from this pool, projects were randomly sampled and subjected to manual build attempts. Projects failing to compile and execute successfully on our environment were excluded. For each language, we selected seven projects. To expand diversity, we selected an additional 10 projects to answer Q3. Hence, we added two extra steps to selected projects. In step three, we manually inspected the Dockerfiles in the projects or DockerHub [12] and deleted those that did not include them. In step four, we manually built images using the Dockerfile in the project or DockerHub and removed the projects that could not build images. We selected 10 projects in Q3. Table 2 displays the important information about these projects. The projects selected come from a variety of fields, such as compilers, natural language analysis tools, interpreters, databases, and firewalls.

Threshold. We set 30 minutes as the threshold. DRAFT completes generating the Dockerfile in 30 minutes is considered a successful generation (✓), and more than 30 minutes of repairs are considered a failed generation (✗). The design of DRAFT allows it to keep trying to repair Dockerfiles until they are successfully built. However, in practice, practitioners need to consider the time consumption. In addition, we use test cases for each project to evaluate further the functionality of the generated images for each successfully generated Dockerfile (test failure ✓). As shown in Table 2, we attempted to use LLMs to generate test cases first, which applied to 18 projects. Second, we looked for test cases in the project documentation, which worked for 10 projects. Finally, we found test cases for the remaining three projects in GitHub Actions.

Approach application methodology. For evaluation fairness, DRAFT and Copilot initially use an identical prompt structure, with the only difference being that the Copilot prompt excludes pre-processing details. The prompt is structured as: project repository link, project language category, instructions to generate Dockerfile commands, and include/exclude project context. For other tools, we supplied the required inputs for each tool according to its requirements. We evaluated whether the generated Docker images can be run successfully. Specifically, we automatically built Dockerfiles generated from each tool and manually verified that the generated images could be successfully executed. If the tool generated a Docker image, we verified that the image can be successfully executed. We confirmed the images in two steps. In the first step, we confirmed that the image had run successfully by checking the state of the container corresponding to each image. In the second

step, we confirmed the artifacts in the image by performing simple tests, *e.g.*, using commands to instruct the software to output its version number.

Experiment environment. We carried out the evaluation on a Linux server running Ubuntu 22.04 with an Intel(R) Xeon(R) Platinum 8163 CPU@2.50GHz, 96 cores, and 376GB memory. To eliminate experimental bias caused by fluctuations in device performance, we conducted three experiments and calculated the arithmetic median for each Dockerfile generation time consumption.

4.2 Effectiveness (Q1)

In this section, we evaluated whether DRAFT can successfully generate Dockerfiles.

Methodology. To answer Q1, we selected 35 mature projects. We selected 7 projects for each of the 5 program languages that our implementation of DRAFT supports (*i.e.*, C/C++, Java, Go, Python, and Ruby). We evaluated the effectiveness of DRAFT using two different LLMs (*i.e.*, ChatGPT 3.5 and Deepseek v3). We also compared DRAFT to language-agnostic tools, such as Copilot (based on ChatGPT 4.0) [9] and Actions_remaker [1], by instructing the two tools to generate a Dockerfile or Docker image. Meanwhile, we compared DRAFT to the tools that can generate Dockerfiles or Docker images for specific languages, such as Repo2docker [31], Boxing [4], and Starter [55]. We checked if these Dockerfiles and images can be run successfully. In addition, we submitted DRAFT-generated Dockerfiles to the project maintainers via pull requests. Table 2 shows the successfully built Dockerfiles or images in each tool column.

Results. From Table 2, DRAFT successfully generated Dockerfiles for 28 projects using ChatGPT, while Dockerfiles generated for 3 projects failed testing, and generation failed for 4 projects. Using DeepSeek, DRAFT successfully generated Dockerfiles for 28 projects, but Dockerfiles generated for 2 projects failed testing, and generation failed for 4 projects. Overall, DRAFT's performance showed slight variations between the two LLMs, with DeepSeek demonstrating marginally better results. Copilot successfully generated Dockerfiles for 11 projects and failed for 24 projects, with a success rate of 31.43% (11/35). Actions_remaker successfully built Docker images for 13 projects, failed for 21 projects, and did not support 1 project, with a success rate of 38.23% (13/34). Repo2docker successfully generated Docker images for 5 projects, with a success rate of 85.71% (6/7). Boxing successfully generated Dockerfiles for 1 project, with a success rate of 14.27% (1/7). Starter failed to generate Dockerfiles for all subjects.

Merged and Rejected Dockerfiles. We submitted 25 pull requests. Tmux maintainers, SQLite users, and MeloTTS users have provided positive feedback. Specifically, SQLite users have found our Dockerfile to be effective in helping developers working on the project for the first time. MeloTTS users have referenced our Dockerfile to resolve an issue with some component version mismatches. Tmux maintainers have merged our Dockerfile into the build instructions. Based on the feedback, we observe the challenges of creating working Dockerfiles, and DRAFT provides developers who are unfamiliar with Docker with an effective method of generating Dockerfiles,

Table 2: Subjects Important Information and Build Results of Generated Dockerfiles (Success (✓), Test Failure (✓), Failure (✗)).

Language	Name	Commit/Tag ID	Stars	Cases Source	DRAFT			Repairs	Copilot	Remaker*	Repo2*	Boxing	Starter
					ChatGPT	Repairs	Deepseek						
C	cpython	3.13.0a4	58.8k	LLMs	✓	6	✓	2	✓	✗	-	-	-
C	flipperzero	0.98.3	10.6k	Doc	✓	1	✓	26	✗	✓	-	-	-
C	CUBRID	11.3	255	-	✗	-	✗	-	✗	✓	-	-	-
C	redis	7.2.4	63.6k	LLMs	✓	0	✓	0	✓	✓	-	-	-
C	sqlite	3.45.1	5.3k	LLMs	✓	4	✓	2	✓	✗	-	-	-
C	tinyc	0.9.27	1.8k	LLMs	✓	36	✓	2	✓	✗	-	-	-
C	tmux	3.4	32.4k	LLMs	✓	1	✓	1	✗	✗	-	-	-
Go	pocketbase	0.22.3	32.1k	Action	✓	24	✓	3	✗	✗	-	-	-
Go	cli	2.43.1	34.9k	LLMs	✓	2	✓	1	✗	✓	-	-	-
Go	OpenGFW	0.2.0	6.6k	LLMs	✓	4	✓	4	✗	✗	-	-	-
Go	sing-box	1.8.5	12.5k	LLMs	✓	0	✓	0	✗	✗	-	-	-
Go	templ	0.2.543	5.9k	Doc	✓	10	✓	0	✗	✗	-	-	-
Go	thanos	0.34.0	12.5k	LLMs	✓	2	✓	0	✗	✓	-	-	-
Go	caddy	v2.9.1	61.5k	Doc	✓	0	✓	1	✗	✗	-	-	-
Java	CoreNLP	4.5.6	9.4k	Doc	✓	2	✓	0	✓	✗	-	-	-
Java	nacos	2.3.0	28.7k	LLMs	✓	0	✓	0	✓	✗	-	-	-
Java	flink	2.0.0-rc0	24.4k	LLMs	✓	15	✓	6	✗	✗	-	-	-
Java	spring-ai	0.8.0	1.2k	Doc	✓	38	✓	0	✓	✓	-	-	-
Java	tutorials	a2b93f3	35.7k	Doc	✓	1	✓	0	✓	✗	-	-	-
Java	conductor	3.17.0	6k	-	✗	-	✗	-	✗	✓	-	-	-
Java	language-tool	6.3	11.4k	Doc	✗	-	✓	2	✗	✓	-	-	-
Python	MeloTTS	0.1.2	2.5k	Doc	✓	0	✓	0	✗	-	✓	-	-
Python	biopython	183	4.1k	LLMs	✓	0	✓	0	✗	✗	✓	-	-
Python	dspy	0c1d1b1	8k	LLMs	✓	1	✓	0	✓	✗	✓	-	-
Python	fastai	2.7.14	25.4k	LLMs	✓	17	✓	1	✗	✓	✓	-	-
Python	GLIP	9dda955	1.9k	LLMs	✓	24	✓	12	✗	✗	✗	-	-
Python	gpt4free	0.2.1.3	54.3k	LLMs	✓	0	✓	0	✓	✓	✓	-	-
Python	tinygrad	0.8.0	22.7k	LLMs	✓	1	✓	1	✗	✗	✓	-	-
Ruby	explore	b3a2fae	4.1k	Doc	✓	0	✓	2	✓	✗	-	✓	✗
Ruby	maybe	7f2633f	25.8k	Doc	✓	22	✓	23	✗	✗	-	✗	✗
Ruby	rails	3.1.2	7.8k	LLMs	✓	1	✓	9	✗	✓	-	✗	✗
Ruby	shopify	13.4.0	1k	Action	✓	0	✓	0	✗	✗	-	✗	✗
Ruby	sidekiq	7.2.1	12.9k	Action	✓	1	✓	0	✓	✓	-	✗	✗
Ruby	solid	2a52568	1.3k	Action	✓	0	✓	1	✗	✗	-	✗	✗
Ruby	mastodon	4.2.5	45.6k	-	✗	-	✗	-	✗	✓	-	✗	✗
Sum					80%		80%		31.43%	38.23%	85.71%	14.27%	0%

✗: This tool does not support generating a Dockerfile for this project Doc: Project documentation Remaker: Action_Remaker Repo2: Repo2Docker

which automatically generates a Dockerfile for a given project. The developers decided against merging 4 pull requests, mainly because the generated Dockerfile needs to be optimized for size, which motivated our enhancement of DRAFT by also reducing image size.

Interpretation. The ability of DRAFT to effectively generate Dockerfiles lies in the automatic prompt generation, which iteratively locates errors in the Dockerfile and drives the LLM to generate patches. To ensure an effective repair, DRAFT identifies whether a patch can repair the error and selects the patch with the highest probability of repairing the error to try. For example, an LLM generates different patches when faced with the same error, and some of them fail. DRAFT generates multiple patches using automatic prompt generation and selects among them, based on rules, the patch most likely to repair the error. Thus, DRAFT is still more effective even with an older LLM version (ChatGPT 3.5) than a newer version of the same LLM (ChatGPT 4.0).

Failed generation. DRAFT fails to generate Dockerfile on 4 projects. We analyzed the error logs and found that the failures came from dependency installation failures and Debian [10] source update failures. Fig 6 outlines some failed project error logs. The *CUBRID* project build relies on *dtrace* [15], but *dtrace* is a tracking tool for Solaris [52] and some Unix-like systems, and does not

```

1. // CUBRID:
2. The dtrace tool was not found. check PATH or install dtrace
3. // Mastodon:
4. RUN curl -fsSL... [signed-by=/etc/apt/keyrings/nodesource.gpg]
   https://deb.nodesource.com/node_20.x
5. Can't create /etc/apt/keyrings/nodesource.gpg: No such file or directory
6. // Conductor:
7. Hit:1 http://security.debian.org/debian-security buster/updates InRelease

```

Figure 6: Error Logs in Projects

usually appear in the base image of most Linux systems. Hence due to dependency installation failure (*dtrace*), DRAFT fails to generate a Dockerfile for *CUBRID*. A similar reason is also found in the *mastodon* project which is unable to generate a Dockerfile due to Node.js [41] installation failure. Repairing them could be done by specifying the mirror source that contains *dtrace* as well as the Node.js mirror source. Debian source update failure is another major cause of DRAFT failure (in *conductor* project), which is usually caused by repository information changes that cause the Dockerfile to build to be unable to update the information properly. The command (`apt-get update`) updates source information, which is usually used in the Dockerfile and can be changed to `apt-get -allow-releaseinfo-change-suite update` to repair this error. In addition, the root cause of test failures lies in LLM-generated

Table 3: Time Consumption (Sec.) and Cost (\$) for DRAFT

Name	Pre	Interaction	Build	Sum	Cost
shopify	2.60	2.36	0.64	5.60	0.01
explore	2.12	2.01	2.02	6.15	0.01
sing-box	3.37	2.54	0.38	6.29	0.01
solid	5.77	2.79	0.07	8.63	0.77
gpt4free	4.53	2.44	2.84	9.81	0.01
candy	4.87	4.60	2.80	12.27	0.20
biopython	7.54	2.32	3.36	13.22	0.03
nacos	8.49	4.59	1.76	14.84	0.03
tinygrad	5.91	9.05	1.02	15.98	0.02
sidekip	2.97	13.58	0.50	17.05	0.01
tmux	9.75	10.48	1.02	21.25	0.02
redis	14.24	2.99	4.13	21.36	0.04
dspy	8.78	12.18	3.79	24.75	0.02
rails	4.35	19.06	1.53	24.94	0.02
MeloTTS	23.37	5.55	0.69	29.61	0.03
thanos	9.12	14.36	10.30	33.78	0.10
cli	12.81	18.69	2.36	33.86	0.22
OpenGFW	11.22	31.18	0.29	42.69	0.36
flipperzero	14.82	34.40	4.18	53.40	0.03
tutorials	14.66	11.57	33.49	59.72	0.84
CoreNLP	12.49	24.81	35.41	72.71	0.10
sqlite	16.93	47.88	33.96	98.77	0.02
GLIP	46.35	94.90	3.02	144.27	0.73
cpython	48.00	144.77	49.86	242.63	0.38
pocketbase	49.32	164.50	32.18	246.00	0.31
maybe	6.36	318.83	1.80	326.99	0.66
fastai	51.20	180.21	255.49	486.90	0.04
templ	147.05	506.17	4.95	658.17	0.38
flink	168.21	441.21	83.39	692.81	0.83
spring-ai	163.51	556.67	21.20	741.38	1.01
tinycc	264.36	895.99	13.73	1,174.08	0.59
Median	12.81	14.36	3.02	33.78	0.04
Average	41.83	131.29	22.57	195.69	0.25

--: Not data Pre: Pre-processing

Dockerfiles' tendency to bypass test execution during build (e.g., using `-DskipTests` in Maven projects), masking dependency resolution issues that subsequently manifest as test failures.

Answer to Q1: DRAFT successfully generated Dockerfiles for 80% (28/35) of projects, substantially outperforming Copilot (31.43%, 11/35) and Actions_remaker (38.23%, 13/34). It also exceeded the performance of specialized tools, including Repo2docker, Boxing, and Starter, despite their language-specific focus. This demonstrates DRAFT's superior effectiveness in Dockerfile generation among all evaluated tools.

4.3 Efficiency (Q2)

In this section, we calculated the time consumption of DRAFT at each stage of generating Dockerfiles.

Methodology. To answer Q2, we evaluated the time consumption of DRAFT in generating Dockerfiles by using ChatGPT. In this evaluation, we did not include projects that failed to successfully build in Q1. Each experiment was performed three times and we recorded the median execution times. The results are shown in Table 3. Each column in the table respectively indicates the time consumption of pre-processing, interaction (prompting LLM and waiting for its response), building of Dockerfiles, and the sum of the above.

Results. From Table 3, we observe that the cumulative time consumed by DRAFT in generating Dockerfiles ranges from 5.60s

(shopify) to 1,174.07s (19.57 minutes, tinycc), with an average of 195.69 and a median of 33.78s. DRAFT required less than 10 repairs to successfully generate Dockerfiles in 77.42% (24/31) of projects. Among these, the initial Dockerfiles generated by DRAFT in 35.48% (10/31) of the projects can build successfully and Copilot only successfully generates Dockerfiles for 4 projects. The most time-consuming part of DRAFT is interaction, with an average of 67.09% (131.29/195.69) total time and a median of 42.51% (14.36/33.78). The median cost of DRAFT using the ChatGPT (gpt-3.5-turbo-1106) is 0.04\$, the total is 7.83\$. The cost of DRAFT using the Deepseek-v3 is 4.18\$. Overall, DRAFT creates Dockerfiles quickly and even the maximum time of 19.57 minutes might be acceptable in many cases as a manual trial-and-error approach could take similarly long or longer.

Interpretation. It is reasonable that interaction with the LLM is the most time-consuming part of DRAFT. This is because automatic prompt generation in DRAFT requires multiple interactions with the LLM, including instructions for generating patches (Dockerfiles) and selecting patches. Builds are usually considered time-consuming, but in DRAFT builds take up an average of 11.53% (22.57/195.69) of the cumulative time and a median of 8.94% (3.02/33.78). DRAFT caches Docker images to accelerate the build of Dockerfiles and therefore consumes less time on the build.

Answer to Q2: Over the subjects, the time consumption of DRAFT in generating Dockerfiles by using ChatGPT lie in 5.60s–19.57 minutes (with a median of 33.78s), the median cost of DRAFT using an LLM is 0.04\$, which demonstrates the efficiency of DRAFT.

4.4 Effectiveness of Components (Q3)

The main components of DRAFT are the pre-processing method and automatic prompt generation. In this section, we explore whether these two approaches are effective separately. We performed two experiments.

Methodology. In the first experiment, we explored whether the pre-processing method used in DRAFT can help Copilot generate Dockerfiles. In the six projects where the initial DRAFT Dockerfile generation succeeded but Copilot failed, we used the same pre-processing method as DRAFT and let Copilot try to generate the Dockerfile based on it. That is, we include the information from the preprocessing method in the prompts that we give Copilot. We verified that the newly generated Dockerfile can be built successfully.

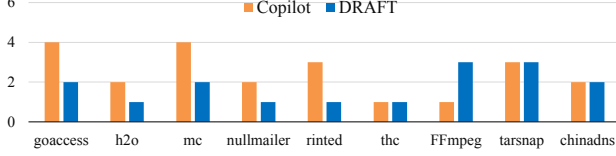
In the second experiment, we explored whether the automatic prompt generation of DRAFT can effectively help LLMs repair Dockerfiles. Therefore, we compared the number of times they repair the same Dockerfile. In this experiment, we refer to the experiments of Lyu et al [38] to increase the diversity of experiments, randomly deleted three dependencies (dependencies of apt install in user-developed Dockerfiles) from nine user-developed Dockerfiles. We inputted the whole build log and Dockerfile for each build failure and instructed it that the Dockerfile had an error and needed to be repaired.

4.4.1 Effectiveness of the Pre-processing Method. To answer this question, on 6 projects (from Q1), we used the same pre-processing information as DRAFT for Copilot and checked whether the Dockerfile generated by Copilot could be built successfully. The results

Table 4: Results of Using the Pre-processing Method to Help Copilot Generate Dockerfiles

Language	Name	Bytes*	Q1	Results
Go	sing-box	8,865	✗	✓
Go	caddy	9,298	✗	✓
Python	MeloTTS	7,862	✗	✗
Python	biopython	9,699	✗	✓
Ruby	shopify	10,858	✗	✓
Ruby	solid_queue	9,099	✗	✓

*: Input Copilot byte count after pre-processing.

**Figure 7: Results of Repairing Dockerfiles**

are shown in Table 4. Each column in the table respectively indicates the project language, name, the number of bytes input to Copilot, the results of Copilot’s Dockerfile in Q1, and new results. The inputs for Copilot are structured as, project repository link, project language category, and generating a Dockerfile for this repository with preprocessing information.

Results. From Table 4, we observe that using the pre-processing method enabled Copilot to complete Dockerfile generation in five (5/6) projects. DRAFT identifies the instruction files (e.g., Readme) in pre-processing. In the instruction files, the project maintainers usually describe how to build the project. This information helps other users to build the software. In Q1, Copilot did not find these key files based on the repository address, thus the Dockerfiles it generated for these projects failed to build. The pre-processing method in DRAFT helps Copilot to obtain this information correctly, which allows it to complete the Dockerfile generation.

Answer to Q3.1: The pre-processing method enabled Copilot to successfully generate 83.33% (5/6) of projects, which demonstrates its effectiveness.

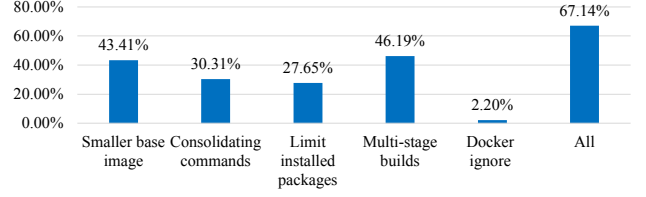
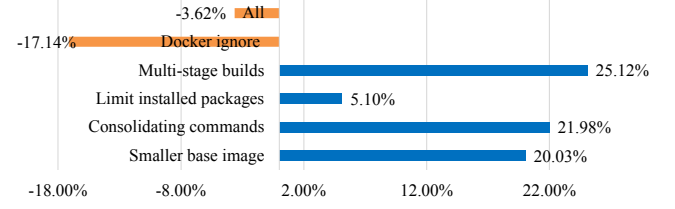
4.4.2 Effectiveness of Automatic Prompt Generation. To answer this question, we let DRAFT and Copilot repair the same Dockerfiles (from user-developed Dockerfiles) and compare the results of the repairs. The inputs for Copilot are structured as a project repository link, a Dockerfile with errors, and the last 50 lines of the build log.

Results. As shown in Fig 7, DRAFT has fewer repairs than Copilot in five projects, an equal number of repairs in three projects, and more repairs than Copilot in one project. DRAFT uses automatic prompt generation to be able to locate errors and provide project context to repair errors.

Answer to Q3.2: The fact that DRAFT can use a lower version of ChatGPT (3.5) in fewer repairs than using a higher version (ChatGPT 4.0) of Copilot demonstrates its effectiveness.

4.5 Dockerfiles Optimization (Q4)

To answer this question, we calculated the difference in image size and build time of Dockerfiles optimized using each of the five methods (see Table 1) relative to the original Dockerfiles.

**Figure 8: Average Reduction in Image Size for Each Method****Figure 9: Average Reduction in Build Time for Each Method**

Methodology. We count how much image size and build time each method optimizes. The method for verifying artifacts is consistent with Section 4.2. In addition, we explore the extent to which these five methods can reduce image size and build time. The results are shown in Fig 8 and Fig 9.

Results. In 27 projects, using DRAFT is effectively optimizing Dockerfiles. From Fig 8, we can observe that “multi-stage builds” (46.19%), “smaller base image” (43.41%), “consolidating commands” (30.31%), and “limit installed packages” (27.65%) can effectively reduce the image size, among which, the multi-stage builds is the most effective. However, in terms of build time, using “Docker ignore” is the most effective way to reduce the time by 17.14%, all other methods increase the build time. The combined use of all methods was able to reduce size by 67.14% and build time by 3.62%. Combining all methods to optimize Dockerfiles cost by a total of 1.57\$ for all projects.

Answer to Q4: In the subjects, using DRAFT can reduce image size by 67.14% and build time by 3.62%, which demonstrates that DRAFT can optimize Dockerfiles.

5 Discussion

The impact of sparse or highly detailed project context on DRAFT. DRAFT overcomes challenges in automatic Dockerfile generation caused by the high variability of modern project documentation. We calculated the median number of fixes required for DRAFT to generate Dockerfiles in different types of documents. As shown in Table 5, documents with < 150 lines required 1 repair from ChatGPT and 0 repairs from Deepseek. Documents with 150–300 lines required 2 repairs from ChatGPT and 1 repair from Deepseek. Documents with > 300 lines required 0 repairs from ChatGPT and 1 repair from Deepseek. As the document size increases, the number of repairs made by DRAFT to generate Dockerfiles does not change significantly, demonstrating that DRAFT is not affected by the richness or sparseness of project documentation.

Error-driven prompts alone are insufficient. DRAFT employs both error-driven prompts and context-driven prompts for error repair

Table 5: Relationship Between the Median Repairs and Document Size

Document Size	Project Number	Median Lines	Repairs ChatGPT	Repairs Deepseek
<150	15	105	1	0
150–300	11	201	2	1
>300	5	450	0	1

1. # Generating Dockerfile	6. # Context
2. Dockerfile_A pip install --no-cache-dir -r requirements.txt	Context from Readme Biopython requires NumPy ... Installation From Source, ... pip install -e . && python setup.py test && sudo python setup.py install
2. Error: no such file or directory: 'requirements.txt'	
3. # Error-driven	7. # Context-driven
4. Dockerfile_B RUN pip install --no-cache-dir .	8. Dockerfile_C RUN pip install --no-cache-dir numpy RUN python setup.py install
5. Error: legacy-install-failure	

Figure 10: Error-driven Prompts and Context-driven Prompts

due to their complementary strengths and limitations. While *error-driven prompts* directly address build errors, they are constrained by incomplete or misleading error logs, as highlighted by Lou et al. [37]. As shown in Figure 10, in generating a Dockerfile for *biopython*, *error-driven prompts* resolved a non-existent “requirements.txt” error by removing the line referencing it, but failed to install essential dependencies like NumPy, leading to continued build failure. In contrast, *context-driven prompts* incorporate project-specific details (e.g., *biopython*’s reliance on setup.py and NumPy [43]) to generate a functional Dockerfile. However, this method risks introducing redundant information for projects with less complex dependencies.

The effect of existing Dockerfiles on DRAFT’s performance. We addressed two potential concerns regarding biasing the results through projects already containing Dockerfiles. First, we ensured that DRAFT did not make use of Dockerfiles included in the project by excluding these Dockerfiles and instructing DRAFT to generate a Dockerfile without them. Second, we addressed the concern of the LLMs being trained on the relevant Dockerfiles, meaning that they could potentially replicate them even if the Dockerfiles are not directly visible to DRAFT.

Regarding the first threat, we did not observe significant performance differences when removing the Dockerfiles from the projects. Despite 28.57% (10/35) of evaluation projects containing Dockerfiles, their removal had no impact on DRAFT’s 80% success rate. This demonstrates performance originates from DRAFT’s self-correction capability, not replication of existing patterns.

Regarding the second threat, we quantified potential bias via text similarity analysis between DRAFT-generated and developer-authored Dockerfiles [50], revealing negligible alignment (average: 0.192, median: 0.178). These statistically insignificant similarity scores, coupled with consistent success rates regardless of Dockerfile presence, confirm DRAFT’s generalizability to projects without pre-existing Dockerfiles. The tool’s performance remains mechanism-driven rather than data-dependent.

Differences between DRAFT’s Dockerfile and User-developed Dockerfile. Key differences in dependency management and image

1. FROM debian:stable	1. FROM alpine:3
2. ...	2. ARG RINETD_VERSION=0.73
3. RUN apt-get update && apt-get install -y \	3. ARG RINETD_URL=https://github.com/samhocevar/rinetd/releases/download/v\${RINETD_VERSION}/rinetd-\${RINETD_VERSION}.tar.gz
4. build-essential \	4. RUN set -xe && apk update \
5. git \	5. && apk add --no-cache build-base curl tar
6. autoconf \	6. && make autoconf m4 perl \
7. automake \	7. && mkdir rinetd && cd rinetd \
8. libtool	7. && curl -sSL \$RINETD_URL tar xz --strip 1 \
9. RUN git clone https://github.com/samhocevar/rinetd.git .	8. && ./configure --prefix=/usr --sysconfdir=/etc/rinetd \
10. && git checkout v0.73	9. && make \ make install \
11. RUN ./configure --prefix=/usr --sysconfdir=/etc/rinetd	10. && cd .. rm -r rinetd \
12. RUN make \ && make install	11. && apk del build-base curl tar automake autoconf m4 perl
13. ...	12. CMD ["rinetd", "-f", "-c", "/etc/rinetd/rinetd.conf"]
14. CMD ["/usr/sbin/rinetd", "-f", "-c", "/etc/rinetd/rinetd.conf"]	

Figure 11: Dockerfile Differences between DRAFT’s Dockerfile (Orange) and User-developed Dockerfile (Blue) in Rinted Project. Differences Are Marked Red

optimization emerge between the DRAFT-generated and user-developed Dockerfiles. Of the nine projects, 66.67% utilized a larger base image for DRAFT, 22.22% employed the same base image, and 11.11% adopted a smaller one. As shown in Figure 11, DRAFT uses a comprehensive Debian [10] base image, including m4 [32] and Perl [46] by default, and adds libtool [34] (orange lines 1–9), avoiding manual installation. Conversely, the user’s Alpine-based file (blue lines 1–3) necessitates explicit dependency installation but achieves a smaller image size by removing these dependencies post-build (blue lines 10–11).

Complexity of the generated Dockerfiles. Quantitative analysis of Dockerfiles shows that the build targets are challenging. During Q1, the generated Dockerfiles and optimized Dockerfiles have an average of 21 and 28 lines of statements, respectively, with median values of 20 and 27. The build instruction files average 207 lines, with a median of 153. In Q3, developer-authored Dockerfiles average 44 lines of statements (median 37), while DRAFT-generated Dockerfiles average 38 lines (median 38). The build instruction files here average 140 lines, with a median of 90. Metrics such as image size and build time further validate the actual complexity of the projects: the generated images average 1.98GB (median 1.03GB), and the build process takes an average of 394.84s (median 172.65s). These figures demonstrate that both automatic and manual build files are complex.

Threats to validity. The main threat to the internal validity of our evaluation is fluctuations in the performance of experimental equipment. To minimize randomness, we stopped all other user processes on the system and conducted three experiments for each project. The results are the median time spent in each phase. In addition, we calculated the standard deviation of the total time spent in three experiments for all projects. We set a threshold that the standard deviation of each project could not exceed 0.5% of the median time. If the standard deviation is exceeded, the results of the experiment are nullified, and the experiment will be repeated.

The main threat to the external validity of our evaluations is the selected experimental projects. One potential threat is how to generalize the results to other projects that we did not consider. To mitigate this risk, we selected a large set of projects. These projects have been thoroughly evaluated and are widely used, showing

that they can be representative of well-known projects developing Dockerfiles for which Dockerfiles would be useful.

The main threat to the construct validity is that the build systems really build the expected artifacts. To make sure that the build system built the expected artifacts, we manually confirmed images and artifacts in two steps. The first step confirms that the image was successfully built and executed by checking the image status in the memory of the experiment machine. The second step confirms the artifact by testing it, *e.g.*, by instructing the software to output the version number. We consider that in our evaluation, build systems successfully build the expected artifacts.

6 Related Work

We introduce the most relevant research related to automatically generating Dockerfiles.

Automatically generating Dockerfiles. Dockerfile automatic generation methods can be broadly categorized into two types. The first type encompasses language-agnostic approaches. Tools like Copilot, an AI-driven and widely adopted solution, offer code completion suggestions as developers type and translate natural language prompts into code recommendations [9]. Leveraging an LLM such as ChatGPT 4.0 [6], Copilot generates Dockerfiles based on user input, supporting multiple programming languages. Nevertheless, its generated Dockerfiles frequently fail to build due to incomplete dependency identification. Rosa et al. [50] introduced a machine learning-based method that employs deep learning to automatically generate Dockerfiles. Their approach trains a model named T5 on a specialized dataset, yet it only achieves a 34% (170/500) build success rate. Similarly, Humpback [27] uses machine learning to enhance Dockerfile development efficiency and reduce errors, but does not support fully automated generation. Starter [55] and Generator-docker [18] support multi-language Dockerfile generation but are no longer actively maintained. The second type includes language-specific methods [4, 44, 47]. DockerizeMe [26] analyzes Python code snippets to infer dependencies by offline extraction of resource and dependency information from the Python Package Index (PyPI [48]), generating a Dockerfile with a repaired base image (Python: 2.7.14). DockerGen [64] constructs a knowledge graph from existing Dockerfiles to facilitate template-based generation. Dockerfile-generator [14] creates Node.js [42] Dockerfiles using user-provided JSON configurations [30].

Both language-agnostic and language-specific methods face limitations in comprehensively identifying dependencies required for successful multi-language project builds. In contrast, DRAFT effectively addresses these challenges by generating robust, buildable Dockerfiles across diverse programming languages.

Maintaining Dockerfiles. The software engineering community has proposed various methods to assist practitioners in maintaining Dockerfiles. Henkel et al. [23] defined development rules to avoid build failures while improving image size and build latency. They also introduced a human-in-the-loop system [24] for detecting and repairing corrupted Dockerfiles. Dockerfile quality can be degraded by smells (*i.e.*, best-practice violations), detected by tools including Hadolint [22], KICS [33], Docker Bench for Security [11], and DockerCleaner [5]. Zhang et al. [65] utilize Dockerfile structural features to recommend appropriate base images. Shabani et al. [51]

proposed an automatic repair framework for Dockerfile flakiness-unpredictable, effectively solving unpredictable temporary build failures caused by external dependencies and constantly changing environments.

Program repair. Program repair techniques can automate the fixing of programming errors [3, 17, 36, 63], with recent research focusing on leveraging LLMs for this purpose [16, 57]. Joshi et al. [29] introduced a multilingual code repair engine powered by code-trained LLMs like Codex [8], which translates natural language into source code. Jin et al. [28] employed a pre-trained LLM for error repair by first fine-tuning it on supervised vulnerability-fixing data, then identifying semantically equivalent vulnerabilities and applying corresponding fixes. Xia et al. [60] evaluated nine LLMs, including Codex, for program repair, highlighting a scaling effect where larger model parameter sizes correlate with better performance. Ribeiro et al. [49] proposed an LLM-based approach to repair type errors in OCaml [45] by enforcing type consistency checks and generating prompts for inconsistencies. ChatRepair [61] interacts with LLMs to fix code errors but lacks error localization capabilities, limiting its applicability to automated Dockerfile generation.

7 Conclusion

This paper has presented a novel approach called DRAFT for generating Dockerfiles through automatic prompt generation. Its underlying core idea is to emulate developers' behavior in iteratively attempting to develop a Dockerfile by utilizing both project documentation and other context as well as error messages that occur upon failed builds by using LLMs. DRAFT is language-agnostic and enables effective and efficient Dockerfile generation for projects by iteratively fixing errors in the Dockerfile. The evaluation results show that DRAFT successfully generates Dockerfiles for 80% of the 35 subjects, with a time consumption of 5.60s–19.57 minutes (with a median of 33.78s). DRAFT (ChatGPT 3.5) has a 51.43% higher success rate in generating Dockerfiles than Copilot, which uses a newer version of LLM (ChatGPT 4.0). In addition, using DRAFT can reduce image size by 67.14% and build time by 3.62%. DRAFT is expected to alleviate practitioners' workloads associated with creating build specifications.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (No.62572237), 2024 Development and Testing Tools Project (CEIEC-2024-ZM02-0066), the Natural Science Foundation of Jiangsu Province (No.BK20241195), and the Innovation Project and Overseas Open Project of State Key Laboratory for Novel Software Technology (Nanjing University) (ZZKT2025A12, ZZKT2025B18, ZZKT2025B20, ZZKT2025B22, KFKT2025A17, KFKT2025A19, KFKT2025A20, KFKT2024A02, KFKT2024A13, KFKT2024A14). This work was also supported by a Singapore Ministry of Education (MOE) Academic Research Fund (AcRF) Tier 1 grant.

References

- [1] Actions-remaker 2025. *Actions-remaker: A Tool to Reproduce Workflow Runs on Github Actions*. Retrieved 2025 from <https://github.com/BugSwarm/actions-remaker>
- [2] Ouafa Bentaleb, Adam S. Z. Belloum, Abderrazak Sebaa, and Aouaouche El-Maouhab. 2022. Containerization technologies: taxonomies, applications and

- challenges. *J. Supercomput.* 78, 1 (2022), 1144–1181. <https://doi.org/10.1007/s11227-021-03914-1>
- [3] Berkay Berabi, Jingxuan He, Veselin Raychev, and Martin T. Vechev. 2021. TFix: Learning to Fix Coding Errors with a Text-to-Text Transformer. In *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18–24 July 2021, Virtual Event (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 780–791. <http://proceedings.mlr.press/v139/berabi21a.html>
 - [4] Boxing 2025. *The zero-configuration Dockerfile generator for Ruby*. Retrieved 2025 from <https://github.com/elct9620/boxing>
 - [5] Q. Bui, M. Laukottter, and R. Scandariato. 2023. DockerCleaner: Automatic Repair of Security Smells in Dockerfiles. In *Proceedings of the 2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. <https://doi.org/10.1109/ICSME58846.2023.00026>
 - [6] ChatGPT 2025. *ChatGPT is a free-to-use AI system*. Retrieved 2025 from <https://chat.openai.com>
 - [7] Jürgen Cito, Gerald Schermann, John Erik Wittern, Philipp Leitner, Sali Zumberi, and Harald C. Gall. 2017. An empirical analysis of the docker container ecosystem on GitHub. In *Proceedings of the 14th International Conference on Mining Software Repositories, MSR 2017, Buenos Aires, Argentina, May 20–28, 2017*, Jesús M. González-Barahona, Abram Hindle, and Lin Tan (Eds.). IEEE Computer Society, 323–333. <https://doi.org/10.1109/MSR.2017.67>
 - [8] Codex 2025. *OpenAI Codex*. Retrieved 2025 from <https://openai.com/blog/openai-codex>
 - [9] Copilot 2025. *GitHub copilot: Your AI pair programmer*. Retrieved 2025 from <https://github.com/features/copilot>
 - [10] Debian 2025. *Debian – The Universal Operating System*. Retrieved 2025 from <https://www.debian.org/>
 - [11] Docker Bench for Security 2025. *The Docker Bench for Security is a script that checks for dozens of common best-practices around deploying Docker containers in production*. Retrieved 2025 from <https://github.com/docker/docker-bench-security>
 - [12] Docker Hub 2025. *Docker Hub Homepage*. Retrieved 2025 from <https://hub.docker.com/>
 - [13] Dockerfile 2025. *What is a Dockerfile?* Retrieved 2025 from https://docs.docker.com/develop/develop-images/dockerfile_best-practices/#what-is-a-dockerfile
 - [14] Dockerfile-generator 2025. *Dockerfile Generator for Node.js*. Retrieved 2025 from <https://github.com/tudvari/dockerfile-generator>
 - [15] DTrace 2025. *DTrace by Example*. Retrieved 2025 from <https://www.oracle.com/solaris/technologies/dtrace-tutorial.html>
 - [16] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated Repair of Programs from Large Language Models. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14–20, 2023*. IEEE, 1469–1481. <https://doi.org/10.1109/ICSE48619.2023.00128>
 - [17] Charaka Geethal, Marcel Böhme, and Van-Thuan Pham. 2023. Human-in-the-Loop Automatic Program Repair. *IEEE Trans. Software Eng.* 49, 10 (2023), 4526–4549. <https://doi.org/10.1109/TSE.2023.3305052>
 - [18] Generator-docker 2025. *Yeoman generator for Docker*. Retrieved 2025 from <https://github.com/microsoft/generator-docker>
 - [19] Gists 2025. *About Gists*. Retrieved 2025 from <https://docs.github.com/en/get-started/writing-on-github/editing-and-sharing-content-with-gists/creating-gists#about-gists>
 - [20] GitHub 2025. *GitHub Homepage*. Retrieved 2025 from <https://github.com/>
 - [21] GPL 2025. *GNU General Public License*. Retrieved 2025 from <https://www.gnu.org/licenses/gpl-3.0.html>
 - [22] Hadolint 2025. *Dockerfile linter, validate inline bash*. Retrieved 2025 from <https://github.com/hadolint/hadolint>
 - [23] J. Henkel, C. Bird, S. K. Lahiri, and T. Reps. 2020. Learning from, Understanding, and Supporting DevOps Artifacts for Docker. In *Proceedings of the 2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*. <https://doi.ieeecomputersociety.org/10.1145/3377811.3380-406>
 - [24] Jordan Henkel, Denini Silva, Leopoldo Teixeira, Marcelo d’Amorim, and Thomas Reps. 2021. Shipwright: A human-in-the-loop system for dockerfile repair. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 1148–1160. <https://doi.org/10.1109/ICSE43902.2021.00106>
 - [25] Eric Horton and Chris Parnin. 2018. Gistable: Evaluating the Executability of Python Code Snippets on GitHub. In *Proceedings of the 2018 IEEE International Conference on Software Maintenance and Evolution, ICSME 2018, Madrid, Spain, September 23–29, 2018*. IEEE Computer Society, 217–227. <https://doi.org/10.1109/ICSME.2018.00031>
 - [26] Eric Horton and Chris Parnin. 2019. DockerizeMe: automatic inference of environment dependencies for python code snippets. In *Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, Montreal, QC, Canada, May 25–31, 2019*, Joanne M. Atlee, Tefvik Bultan, and Jon Whittle (Eds.). IEEE / ACM, 328–338. <https://doi.org/10.1109/ICSE.2019.00047>
 - [27] Humpback 2025. *Quickly build lightweight docker cloud for enterprise user*. Retrieved 2025 from <https://humpback.github.io/humpback/#/>
 - [28] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. InferFix: End-to-End Program Repair with LLMs. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3–9, 2023*, Satish Chandra, Kelly Blincoe, and Paolo Tonella (Eds.). ACM, 1646–1656. <https://doi.org/10.1145/3611643.3613892>
 - [29] Harshit Joshi, José Cambrero Sanchez, Sumit Gulwani, Vu Le, Gust Verbruggen, and Ivan Radiček. 2023. Repair Is Nearly Generation: Multilingual Program Repair with LLMs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 5131–5140. <https://ojs.aaai.org/index.php/AAAI/article/view/25642>
 - [30] JSON 2025. *JSON: a lightweight data-interchange format*. Retrieved 2025 from <https://www.json.org/json-en.html>
 - [31] Jupyter-repo2docker 2025. *Jupyter-repo2docker Documentation*. Retrieved 2025 from <https://repo2docker.readthedocs.io/en/latest/index.html>
 - [32] Brian W Kernighan and Dennis M Ritchie. 1977. *The M4 macro processor*. Bell Laboratories Murray Hill, NJ.
 - [33] KICS 2025. *Find security vulnerabilities, compliance issues, and infrastructure misconfigurations with KICS*. Retrieved 2025 from <https://github.com/Checkmarx/kics>
 - [34] Libtool 2025. *GNU Libtool*. Retrieved 2025 from <https://www.gnu.org/software/libtool/>
 - [35] Nándor Licker and Andrew Rice. 2019. Detecting incorrect build rules. In *Proceedings of the 41st International Conference on Software Engineering, 2019, Montreal, QC, Canada, May 25–31, 2019*, Joanne M. Atlee, Tefvik Bultan, and Jon Whittle (Eds.). IEEE / ACM, 1234–1244. <https://doi.org/10.1109/ICSE.2019.00125>
 - [36] Kui Liu, Jingtang Zhang, Li Li, Anil Koyuncu, Dongsun Kim, Chunpeng Ge, Zhe Liu, Jacques Klein, and Tegawendé F. Bissyandé. 2023. Reliable Fix Patterns Inferred from Static Checkers for Automated Program Repair. *ACM Trans. Softw. Eng. Methodol.* 32, 4 (2023), 96:1–96:38. <https://doi.org/10.1145/3579637>
 - [37] Yiling Lou, Junjie Chen, Lingming Zhang, Dan Hao, and Lu Zhang. 2019. History-driven build failure fixing: how far are we?. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2019, Beijing, China, July 15–19, 2019*, Dongmei Zhang and Anders Möller (Eds.). ACM, 43–54. <https://doi.org/10.1145/3293882.3330578>
 - [38] Jun Lyu, Shanshan Li, He Zhang, Lanxin Yang, Bohan Liu, and Manuel Rigger. 2024. Towards Efficient Build Ordering for Incremental Builds with Multiple Configurations. *Proc. ACM Softw. Eng.* 1, FSE (2024), 1494–1517. <https://doi.org/10.1145/3660774>
 - [39] Dirk Merkel et al. 2014. Docker: lightweight linux containers for consistent development and deployment. *Linux j* 239, 2 (2014), 2.
 - [40] NASM 2025. *A Cross-platform X86 Assembler with an Intel-like Syntax*. Retrieved 2025 from <https://nasm.us/>
 - [41] Node.js 2025. *Node.js Documentation*. Retrieved 2025 from <https://nodejs.org/docs/latest-v20.x/api/index.html>
 - [42] Node.js 2025. *Node.js is an open-source, cross-platform JavaScript runtime environment*. Retrieved 2025 from <https://nodejs.org/en>
 - [43] NumPy 2025. *NumPy Documentation*. Retrieved 2025 from <https://numpy.org/doc/stable>
 - [44] Daniel Nüst and Matthias Hinz. 2019. containerit: Generating Dockerfiles for reproducible research with R. *Journal of Open Source Software* 4, 40 (2019), 1603. <https://doi.org/10.21105/joss.01603>
 - [45] OCaml 2025. *OCaml Documentation*. Retrieved 2025 from <https://ocaml.org/docs>
 - [46] Perl 2025. *The Perl Programming Language*. Retrieved 2025 from <https://www.perl.org/>
 - [47] Phpdocker.io 2025. *PHP and docker based generator*. Retrieved 2025 from <https://github.com/phpdocker-io/phpdocker.io/>
 - [48] PyPI 2025. *The Python Package Index*. Retrieved 2025 from <https://pypi.org/>
 - [49] Francisco Ribeiro, José Nuno Castro de Macedo, Kanae Tsushima, Rui Abreu, and João Saraiva. 2023. GPT-3-Powered Type Error Debugging: Investigating the Use of Large Language Models for Code Repair. In *Proceedings of the 16th ACM SIGPLAN International Conference on Software Language Engineering, SLE 2023, Cascais, Portugal, October 23–24, 2023*, João Saraiva, Thomas Degueule, and Elizabeth Scott (Eds.). ACM, 111–124. <https://doi.org/10.1145/3623476.3623522>
 - [50] Giovanni Rosa, Antonio Mastropaolo, Simone Scalabrino, Gabriele Bavota, and Rocco Oliveto. 2023. Automatically Generating Dockerfiles via Deep Learning: Challenges and Promises. In *Proceedings of the 2023 IEEE/ACM International Conference on Software and System Processes (ICSSP)*. 1–12. <https://doi.org/10.1109/ICSSP59042.2023.00011>
 - [51] Taha Shabani, Noor Nashid, Parsa Alian, and Ali Mesbah. 2025. Dockerfile Flakiness: Characterization and Repair. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 1793–1805. <https://doi.org/10.1109/ICSE55347.2025.00238>
 - [52] Solaris 2025. *Oracle Solaris Documentation*. Retrieved 2025 from <https://docs.oracle.com/en/operating-systems/solaris.html>
 - [53] Stack Overflow 2025. *Stack Overflow - Where Developers Learn, Share, & Build Careers*. Retrieved 2025 from <https://numpy.org/doc/stable>
 - [54] StackOverflow 2025. *2022 developer survey*. Retrieved 2025 from <https://survey.stackoverflow.co/2022/#section-most-loved-dreaded-and-wanted-other-tools>

- [55] Starter 2025. *Starter: helping you get started with containerized apps*. Retrieved 2025 from <https://github.com/cloud66-oss/starter>
- [56] TIOBE Ranking 2025. *TIOBE Programming Community index*. Retrieved 2025 from <https://www.tiobe.com/tiobe-index/>
- [57] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the Copilots: Fusing Large Language Models with Completion Engines for Automated Program Repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023*, Satish Chandra, Kelly Blincoe, and Paolo Tonella (Eds.). ACM, 172–184. <https://doi.org/10.1145/3611643.3616271>
- [58] Yiwen Wu, Yang Zhang, Tao Wang, and Huaimin Wang. 2022. Dataset for Characterizing the Occurrence of Dockerfile Smells in Open-Source Software. <https://doi.org/10.21227/r9v8-4f07>
- [59] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 126:1–126:13. <https://doi.org/10.1145/3597503.3639121>
- [60] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated Program Repair in the Era of Large Pre-trained Language Models. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 1482–1494. <https://doi.org/10.1109/ICSE48619.2023.00129>
- [61] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for \$0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, Maria Christakis and Michael Pradel (Eds.). ACM, 819–831. <https://doi.org/10.1145/3650212.3680323>
- [62] YASM 2025. *Yasm Is a Complete Rewrite of the Nasm Assembler under the “new” Bsd License*. Retrieved 2025 from <https://yasm.tortall.net/>
- [63] Michihiro Yasunaga and Percy Liang. 2020. Graph-based, Self-Supervised Program Repair from Diagnostic Feedback. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event (Proceedings of Machine Learning Research, Vol. 119)*. PMLR, 10799–10808. <http://proceedings.mlr.press/v119/yasunaga20a.html>
- [64] Hongjie Ye, Jiahong Zhou, Wei Chen, Jiaxin Zhu, Guoquan Wu, and Jun Wei. 2021. Dockergen: A knowledge graph based approach for software containerization. In *Proceedings of the 2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 986–991. <https://doi.org/10.1109/COMPSAC51774.2021.00133>
- [65] Yinyuan Zhang, Yang Zhang, Xinjun Mao, Yiwen Wu, Bo Lin, and Shangwen Wang. 2022. Recommending Base Image for Docker Containers based on Deep Configuration Comprehension. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2022, Honolulu, HI, USA, March 15-18, 2022*. IEEE, 449–453. <https://doi.org/10.1109/SANER53432.2022.00060>